

Term Project: Instant Messaging App

Architecture Document

Table of Contents

1	Introduction.....	2
1.1	<i>Purpose.....</i>	2
1.2	<i>Target Audience.....</i>	2
1.3	<i>Terms and Definitions.....</i>	2
2	Architectural Goals and Constraints.....	3
2.1	<i>Security.....</i>	3
2.2	<i>Persistency.....</i>	3
2.3	<i>Reliability/Availability.....</i>	3
3	System Overview.....	4
4	System Architecture.....	5
4.1	<i>Front-end Architecture.....</i>	5
4.2	<i>Back-end Microservices Architecture.....</i>	5
5	Database Design.....	6
6	Issue and Concerns.....	7

1 Introduction

This document outlines the architecture for the CS 314 Term Project, which is an instant messaging app similar to Slack or Telegram. The application consists of a chat server and multiple chat clients that allow users to send and receive real-time messages over the Internet.

1.1 Purpose

The purpose of this document is to provide a high-level description of the system architecture to guide developers in understanding the frontend and backend implementations.

1.2 Target Audience

The target audience for this document is Prof. Fei Xie and TA Huan Wu.

1.3 Terms and Definitions

MERN: A technology stack consisting of MongoDB, Express, React, and Node.js.

Ngrok: A tool used to deploy the remote backend service and expose it to the Internet.

JWT: JSON Web Tokens, used for secure user authentication and authorization.

Socket.IO: A library utilized for real-time WebSocket communication and direct messaging.

2 Architectural Goals and Constraints

2.1 Security

The system protects user data from unauthorized access using JSON Web Tokens (JWT). During signup, the system accepts an email and password, hashes the password using bcrypt, and generates a JWT token. This token is stored in a cookie. A verifyToken middleware is implemented to ensure that only requests containing a valid JWT cookie can access the API endpoints.

2.2 Persistency

The system achieves data persistence by using a database to store user credentials, messaging history, and a list of created chat rooms and their participants.

2.3 Reliability/Availability

A major system constraint regarding availability is the reliance on Ngrok for accessing the remote backend. If the frontend is not configured with specific headers ("ngrok-skip-browser-warning": "true") and withCredentials: true, the system cannot behave as expected due to CORS errors and Ngrok security splash pages blocking connections

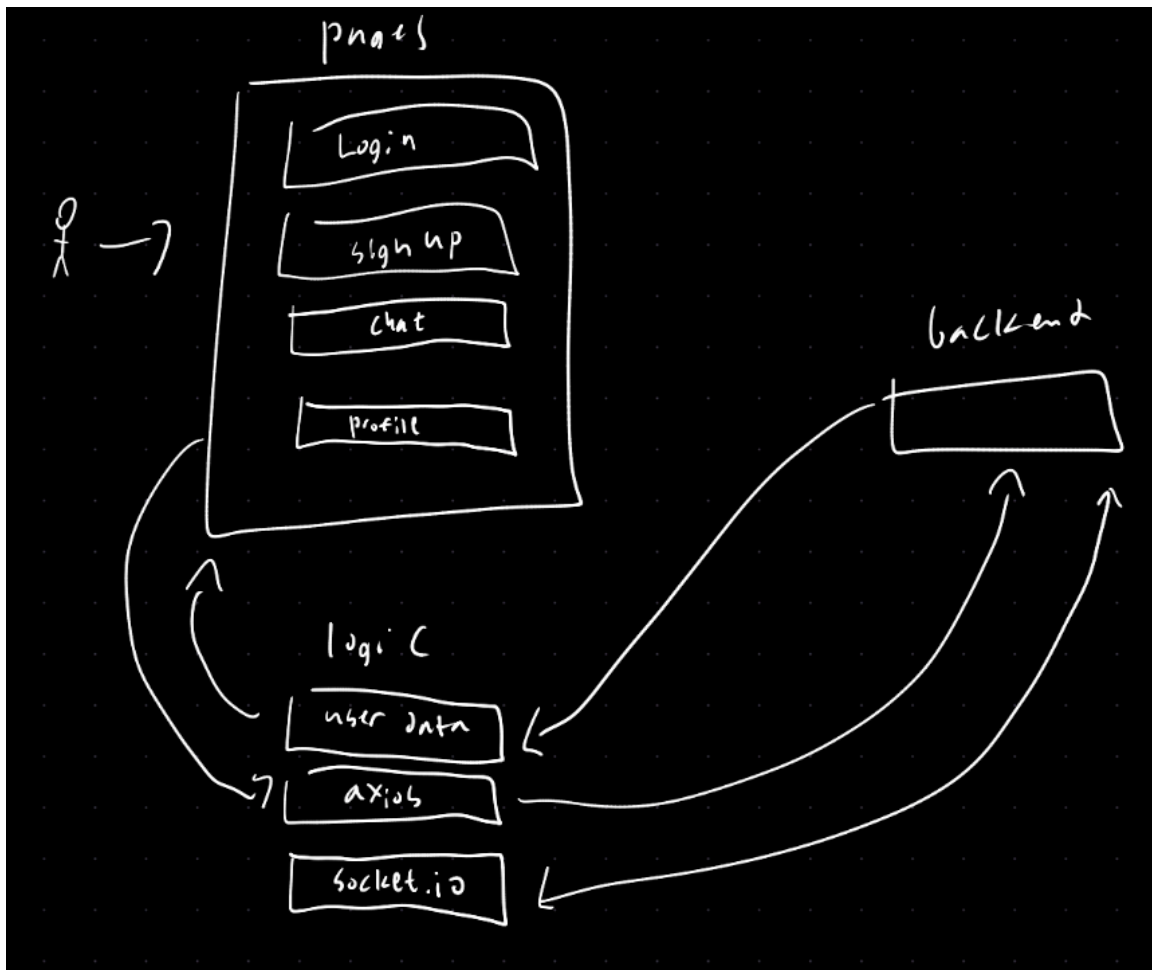
3 System Overview

The application is a client-server instant messaging system. The user interface, built with React, allows users to manage their profiles, authenticate, navigate chat rooms, and view chat history. Application services on the backend handle these authentication workflows, contact management, and direct messaging. Shared infrastructure includes real-time message delivery between clients over the Internet, achieved by WebSockets on the backend. Database management relies on MongoDB to store and retrieve user and message data, which is also hosted on MongoDB Atlas.

4 System Architecture

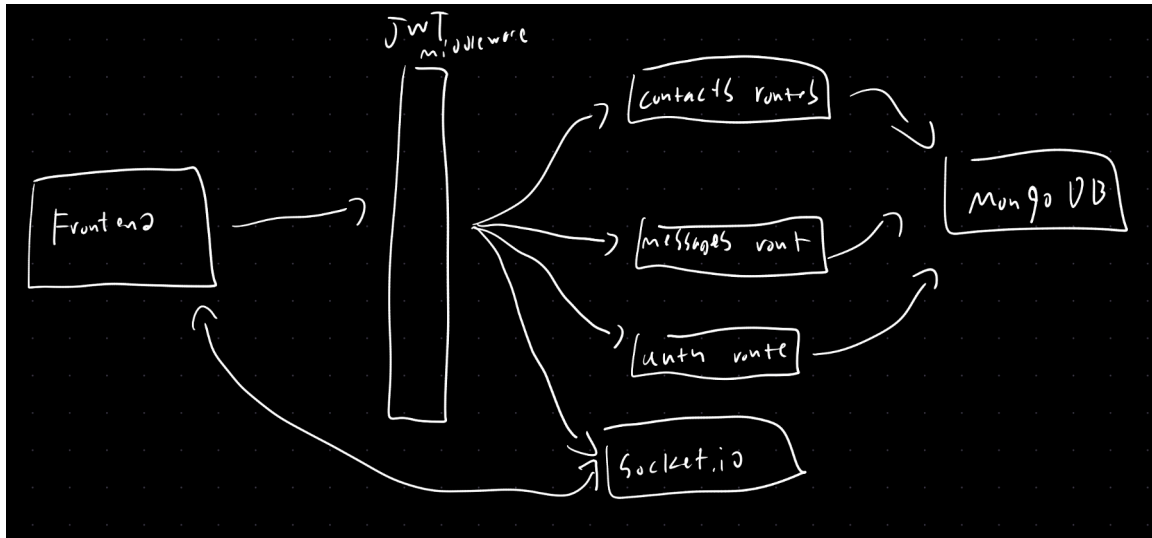
4.1 Front-end Architecture

The frontend utilizes React. It integrates with the backend APIs via Axios. The API client is configured with a baseURL pointing to the remote Ngrok domain, ensuring tokens are included in cross-site requests via withCredentials: true. Real-time chat functionalities are managed using Socket.IO to connect to the backend.



4.2 Back-end Microservices Architecture

The backend utilizes Node.js and the Express web application framework for building RESTful APIs. API routes are logically divided into Auth (/api/auth), Contacts (/api/contacts), and Messages (/api/messages). Real-time service communication is handled via Socket.IO, where the backend maps an incoming user's userId to their socket connection to ensure direct messages are routed to the correct sender and recipient.



5 Database Design

The database of choice is MongoDB, hosted via MongoDB Atlas, which is advantageous for storing and retrieving data efficiently for all features within the MERN stack. The database design supports interoperability by maintaining schema structures for users and messages. The user schema includes fields like id, email, firstName, lastName, image, and color. The message schema stores the sender, recipient, text content, messageType, and a timestamp to ensure accurate delivery and conversation history retrieval. We are using MongoDB.

6 Issue and Concerns

A key technical challenge involves handling cross-origin requests and Ngrok restrictions to avoid CORS errors. On the frontend, developers must ensure that both Axios and Socket.IO configurations explicitly include the "ngrok-skip-browser-warning": "true" extra header and withCredentials: true. Without this, cross-origin cookies, tokens, and WebSocket handshakes will not function smoothly. Correspondingly, the backend must be properly configured to handle CORS to accept these cross-origin requests and credentials from the frontend application. Additionally, error handling must be carefully managed for invalid user actions or database failures.